ALGORITHMIC EFFICIENCY & RECURSION TOOLKIT (AERT)

Course: Data Structures (ETCCDS202)

Unit: Foundations & Algorithmic Analysis

Student Name: Navraj

Roll No: 2501730493

Program: B.Tech CSE (AI/ML – Data Science)

1. INTRODUCTION

This assignment focuses on understanding Abstract Data Types (ADT), recursion, algorithmic paradigms, and complexity analysis.

A Python toolkit named AERT (Algorithmic Efficiency & Recursion Toolkit) was developed to implement and analyze common recursive algorithms such as Factorial, Fibonacci (Naive and Memoized), Tower of Hanoi, and Binary Search.

The main objective is to understand how algorithms behave in terms of time and space efficiency and how different paradigms like Divide & Conquer, Dynamic Programming, and Recursion work in practice.

2. ABSTRACT DATA TYPE (ADT) – STACK

An Abstract Data Type (ADT) defines a data structure logically by specifying what operations can be performed and what behavior is expected, without describing how it is implemented internally.

In this assignment, a Stack ADT is implemented with the following operations:

push(x), pop(), peek(), is_empty(), and size().

All stack operations are O(1) because they operate only on the top element. No traversal or shifting of elements is required, so each operation takes constant time.

3. FACTORIAL USING RECURSION

Factorial of a number n is defined as:

$n! = n \times (n-1)!$ with base case $0! = 1$.

Time Complexity: O(n)

Space Complexity: O(n)

Each recursive call reduces the problem size by 1 and uses the call stack.

4. FIBONACCI SERIES

4.1 Naive Recursive Fibonacci

The Fibonacci sequence is defined as:

$F(n) = F(n-1) + F(n-2)$

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

The naive approach is slow because it recalculates the same subproblems multiple times.

4.2 Fibonacci with Memoization (Dynamic Programming)

In memoization, we store already computed values to avoid repeated calculations.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Memoization makes the Fibonacci algorithm efficient and scalable.

Naive Fibonacci is slow for large n because of repeated recursive calls for the same values.

5. TOWER OF HANOI

Tower of Hanoi is a classic recursive problem where disks are moved from a source rod to a destination rod using an auxiliary rod while following size constraints.

Number of moves required:

Moves = $2^n - 1$

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

The recursion depth equals the number of disks.

6. RECURSIVE BINARY SEARCH

Binary search works on sorted arrays and repeatedly divides the search space into halves.

Recurrence relation:

$T(n) = T(n/2) + O(1)$

Time Complexity:

Best Case: O(1)

Average Case: O(log n)

Worst Case: O(log n)

Binary search handles edge cases like empty array and key not found by returning -1.

7. ALGORITHMIC PARADIGMS

The assignment demonstrates the following paradigms:

Binary Search → Divide & Conquer

Fibonacci (Memoized) → Dynamic Programming

Tower of Hanoi → Recursion

8. REAL LIFE EXAMPLES

Divide & Conquer:

Searching a contact in a phone directory by repeatedly dividing the search space.

Dynamic Programming:

Saving previous calculations in memory like a calculator's memory function.

Greedy Approach:

Giving change using the highest denomination coins first.

9. LEARNING OUTCOMES

Through this assignment, the following concepts were understood:

- Difference between Data Structure and ADT

- Writing correct recursive functions with base cases

- Understanding recursion call stack

- Analyzing time and space complexity using Big-O

- Identifying correct algorithmic paradigm

10. CONCLUSION

The AERT toolkit demonstrates how recursion can be used to solve problems efficiently and clearly.

It also shows that naive approaches can be slow, and optimization techniques like memoization are important for improving performance.

This assignment strengthened the understanding of algorithm design, complexity analysis, and real-world application of data structures.

11. REFERENCES

Class Notes

Data Structures Textbook

Python Documentation